



**Tecnológico
de Monterrey**

**MA2003B Aplicación de Métodos Multivariados
en Ciencia de Datos**

Módulo 2: Relaciones de Dependencia

Profesor: Raul Gomez
Correo: rgomez@tec.mx
Oficina: A4-123A

Tabla de contenidos

I	Análisis de Regresión	3
1.	Método de mínimos cuadrados	5
2.	El modelo de regresión lineal simple	13

Parte I

Análisis de Regresión

Capítulo 1

Método de mínimos cuadrados

Supongamos que tenemos un conjunto de datos

$$((x_1, y_1), (x_2, y_2), \dots, (x_n, y_n))$$

Por ejemplo, consideremos el siguiente conjunto de datos simulado:

```
library(tidyverse)
library(krulRutils)
set.seed(123)
n <- 50
x_data <- runif(n, min = 0, max = 10)
epsilon_data <- rnorm(n, mean = 0, sd = 1)
y_data <- x_data + epsilon_data

sample_data_tbl <- tibble(
  x = x_data,
  y = y_data
)
```

Notemos que podemos visualizar estos datos mediante un diagrama de dispersión:

```
sample_data_plot <- sample_data_tbl %>%
  ggplot(aes(x = x, y = y)) +
  geom_point(
    color = c_pal("C red"),
    size = 1,
    alpha = 1
  ) +
  coord_fixed() +
  labs(
    title = "",
    x = "",
    y = ""
  )
```

```
) +  
theme_krul()  
  
sample_data_plot
```

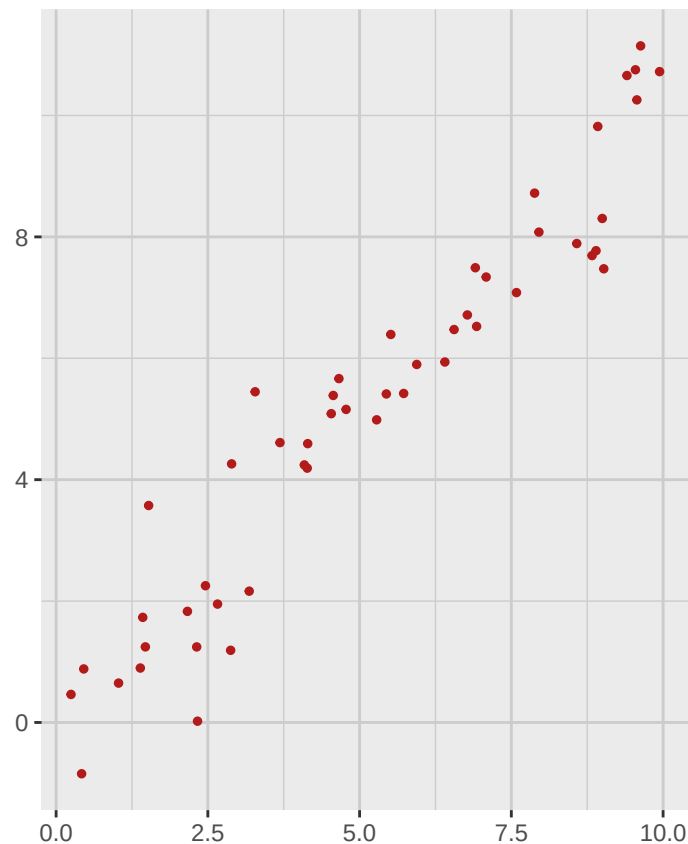


Figura 1.1: Diagrama de dispersión de los datos simulados.

Buscamos ahora generar una línea recta que mejor se ajuste a estos datos. Hay varias formas de definir “mejor ajuste”, pero una de las más comunes es minimizar la suma de los errores al cuadrado (SSE por sus siglas en inglés) entre los valores observados y los valores correspondientes sobre la línea recta propuesta. De manera más precisa, buscamos una recta

$$Y = \hat{f}(X) = \hat{\beta}_0 + \hat{\beta}_1 X$$

tal que si definimos $\hat{y}_i = \hat{f}(x_i)$ entonces minimicemos la suma

$$\text{SSE} = \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Para encontrar la fórmula para $\hat{\beta}_0$ y $\hat{\beta}_1$ en términos de nuestros datos, utilizaremos un poco de álgebra lineal. Empecemos por fijar el *vector de las observaciones independientes*:

$$x = (x_1, x_2, \dots, x_n)$$

y el vector de las observaciones dependientes:

$$y = (y_1, y_2, \dots, y_n)$$

Consideraremos ahora el subespacio

$$W = \{\beta_0 1_n + \beta_1 x \mid \beta_0, \beta_1 \in \mathbb{R}\} \subset \mathbb{R}^n$$

donde $1_n = (1, 1, \dots, 1)$ es el vector en $\mathbb{R}^n$ con todas sus entradas iguales a 1. Notemos que W es un subespacio de dimensión 2 generado por los vectores 1_n y x . Nuestro objetivo es encontrar el vector $\hat{y} \in W$ que esté lo más cercano posible al vector de observaciones dependientes y . En otras palabras, buscamos minimizar el cuadrado de la distancia entre los vectores y e $\hat{y}$:

$$\|y - \hat{y}\|^2 = \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Para lograr esto, utilizaremos la proyección ortogonal. Recordemos que la proyección ortogonal de un vector v sobre un subespacio W es el vector en W que está más cercano a v . En nuestro caso queremos proyectar el vector de observaciones y sobre el subespacio W . Para ello primero necesitamos encontrar una base ortogonal para W , por lo que utilizaremos el proceso de Gram-Schmidt para ortogonalizar los vectores 1_n y x . Definimos entonces los vectores:

$$\begin{aligned} u_1 &= 1_n \\ u_2 &= x - \frac{\langle x, u_1 \rangle}{\langle u_1, u_1 \rangle} u_1 = x - \bar{x} 1_n \end{aligned}$$

donde

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$$

es la media de las observaciones independientes. Como u_1 y u_2 son ortogonales, podemos calcular la proyección ortogonal de y sobre W utilizando la fórmula:

$$\hat{y} = \frac{\langle y, u_1 \rangle}{\langle u_1, u_1 \rangle} u_1 + \frac{\langle y, u_2 \rangle}{\langle u_2, u_2 \rangle} u_2$$

Desarrollando esta expresión, obtenemos:

$$\hat{y} = \bar{y} 1_n + \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2} (x - \bar{x} 1_n)$$

donde

$$\bar{y} = \frac{1}{n} \sum_{i=1}^n y_i$$

es la media de las observaciones dependientes. Finalmente, al comparar esta expresión con la forma de la recta $y = \hat{\beta}_0 + \hat{\beta}_1 x$, podemos identificar los coeficientes de la siguiente manera:

$$\begin{aligned} \hat{\beta}_1 &= \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2} \\ \hat{\beta}_0 &= \bar{y} - \hat{\beta}_1 \bar{x} \end{aligned}$$

Observación 1.0.1. Recordemos que la covarianza muestral entre las observaciones x e y está dada por:

$$s_{xy} = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})$$

y la varianza muestral de las observaciones x está dada por:

$$s_x^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2$$

Por lo tanto, podemos reescribir el coeficiente $\hat{\beta}_1$ como:

$$\hat{\beta}_1 = \frac{s_{xy}}{s_x^2}$$

Para el ejemplo de datos simulados que presentamos al inicio de esta sección, podemos calcular la línea recta que mejor se ajusta a estos datos utilizando la función de regresión lineal en R:

```
fitted_data_tbl <- sample_data_tbl %>%  
  mutate(  
    fitted = fitted(lm(y ~ x, data = .))  
  )  
  
fitted_data_plot <- fitted_data_tbl %>%  
  ggplot(aes(x = x, y = y)) +  
  geom_smooth(method = "lm", se = FALSE, color = c_pal("C blue")) +  
  geom_segment(  
    aes(xend = x, yend = fitted),  
    color = c_pal("C green"),  
    alpha = 1  
  ) +  
  geom_point(color = c_pal("C red"), size = 1, alpha = 1) +  
  geom_point(  
    aes(x = x, y = fitted),  
    color = c_pal("C red"),  
    size = 1,  
    alpha = 1  
  ) +  
  coord_fixed() +  
  labs(  
    title = "",  
    x = "",  
    y = ""  
  ) +  
  theme_krul()  
  
fitted_data_plot
```

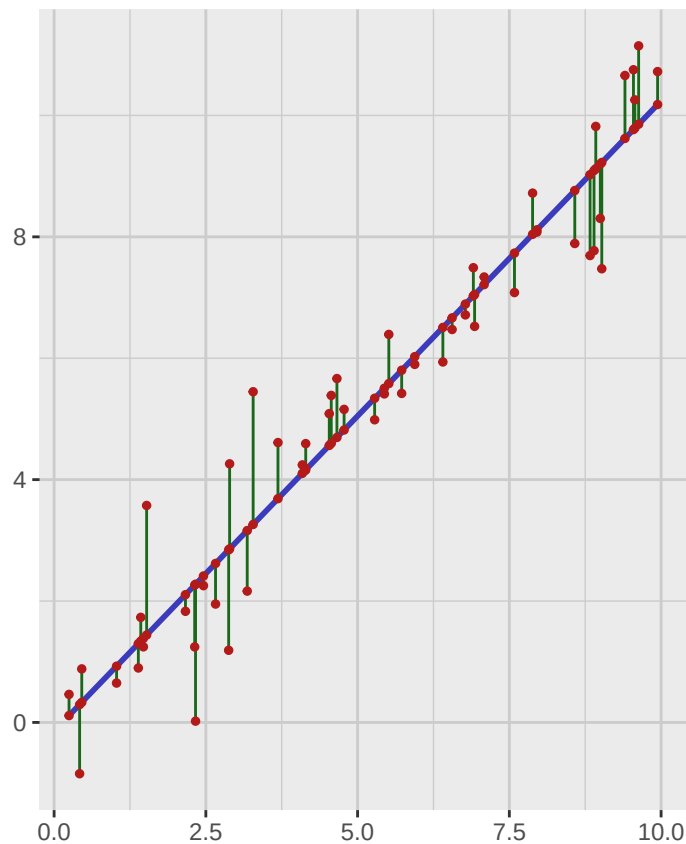


Figura 1.2: Línea recta que mejor se ajusta a los datos simulados.

Notemos que este proceso se puede aplicar a cualquier conjunto de datos dado, sin embargo una de las aplicaciones más comunes de este método es tratar de predecir el valor de la variable Y dado un valor específico de la variable X que no se encuentre en el conjunto de datos original. Se sigue que para que estas predicciones sean de utilidad, es necesario que el comportamiento de nuestros datos siga una tendencia lineal. En caso contrario, las predicciones podrían no ser confiables. Por ejemplo, consideremos el ajuste por el método de mínimos cuadrados para los siguientes conjuntos de datos:

```
library(patchwork)

set.seed(123)

n <- 500
x_data1 <- runif(n, min = 0, max = 10)
epsilon_data1 <- rnorm(n, mean = 0, sd = 50)
y_data1 <- (x_data1)^3 + epsilon_data1

x_data2 <- runif(n, min = 0, max = 10)
epsilon_data2 <- rnorm(n, mean = 0, sd = 0.1)
y_data2 <- sin(x_data2) + epsilon_data2
```

```
x_data3 <- runif(n, min = 0, max = 10)
y_data3 <- runif(n, min = 0, max = 10)

x_data4 <- runif(n, min = 0, max = 10)
y_data4 <- mapply(function(mu, sigma) rnorm(1, mean = mu, sd = sigma),
                  mu = x_data4,
                  sigma = x_data4)

sample_data1_tbl <- tibble(
  x = x_data1,
  y = y_data1
)

sample_data2_tbl <- tibble(
  x = x_data2,
  y = y_data2
)

sample_data3_tbl <- tibble(
  x = x_data3,
  y = y_data3
)

sample_data4_tbl <- tibble(
  x = x_data4,
  y = y_data4
)

sample_data1_plot <- sample_data1_tbl %>%
  ggplot(aes(x = x, y = y)) +
  geom_point(
    color = c_pal("C red"),
    size = 1,
    alpha = 1
  ) +
  geom_smooth(method = "lm", se = FALSE, color = c_pal("C blue")) +
  labs(
    title = "Datos cúbicos",
    x = "",
    y = ""
  ) +
  theme_krul()

sample_data2_plot <- sample_data2_tbl %>%
  ggplot(aes(x = x, y = y)) +
```

```
geom_point(  
  color = c_pal("C red"),  
  size = 1,  
  alpha = 1  
) +  
geom_smooth(method = "lm", se = FALSE, color = c_pal("C blue")) +  
labs(  
  title = "Datos senoidales",  
  x = "",  
  y = ""  
) +  
theme_krul()  
  
sample_data3_plot <- sample_data3_tbl %>%  
  ggplot(aes(x = x, y = y)) +  
  geom_point(  
    color = c_pal("C red"),  
    size = 1,  
    alpha = 1  
  ) +  
  geom_smooth(method = "lm", se = FALSE, color = c_pal("C blue")) +  
  labs(  
    title = "Datos aleatorios",  
    x = "",  
    y = ""  
  ) +  
  theme_krul()  
  
sample_data4_plot <- sample_data4_tbl %>%  
  ggplot(aes(x = x, y = y)) +  
  geom_point(  
    color = c_pal("C red"),  
    size = 1,  
    alpha = 1  
  ) +  
  geom_smooth(method = "lm", se = FALSE, color = c_pal("C blue")) +  
  labs(  
    title = "Datos con varianza creciente",  
    x = "",  
    y = ""  
  ) +  
  theme_krul()  
  
sample_data_plot <- (sample_data1_plot + sample_data2_plot) /  
  (sample_data3_plot + sample_data4_plot)
```

```
sample_data_plot
```

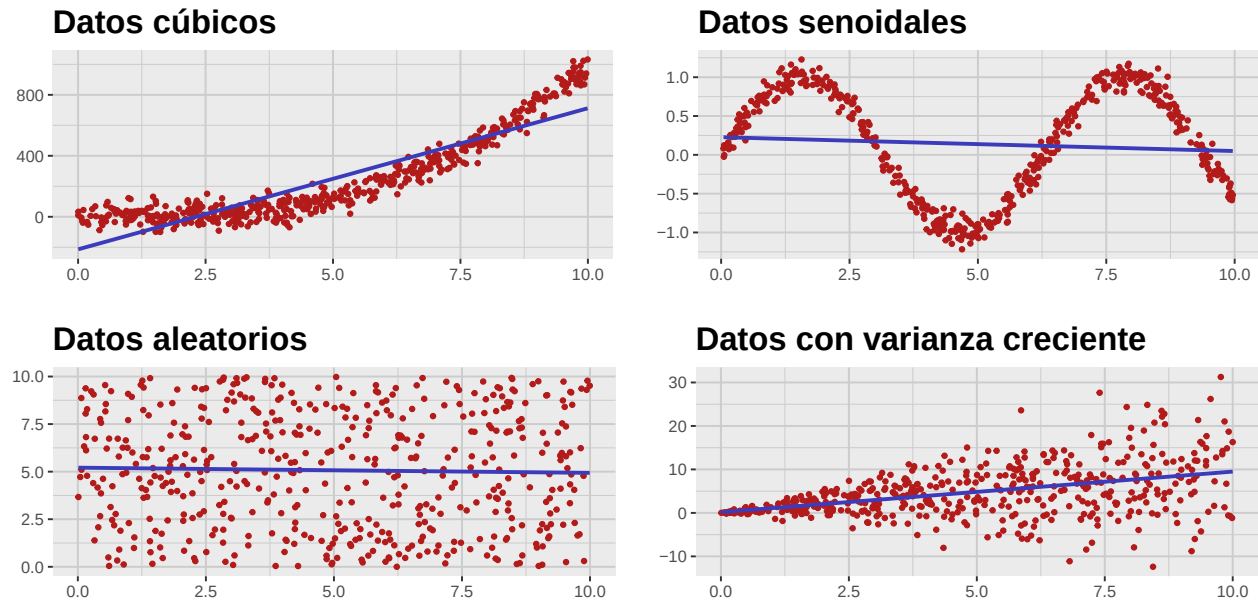


Figura 1.3: Ajuste por mínimos cuadrados para diferentes conjuntos de datos.

Es bastante claro que en estos casos el ajuste por mínimos cuadrados no es el método adecuado para realizar predicciones confiables. En general, este método solo es efectivo cuando se cumplen ciertas hipótesis que estudiaremos en las siguientes secciones, cuando introduciremos el modelo de regresión lineal desde un punto de vista estadístico.

Capítulo 2

El modelo de regresión lineal simple

En un *modelo de regresión simple*, suponemos que tenemos una *variable independiente* X_0 (también conocida como *variable explicativa* o *predictora*), y una *variable dependiente* Y_0 (también conocida como *variable de respuesta*). Suponemos además que estas variables satisfacen una relación de la forma:

$$Y_0 = f_0(X_0) + \epsilon$$

donde f_0 es una función desconocida que describe la relación entre ambas variables, y ϵ es una variable aleatoria que representa el *error* o *ruido* en la relación entre X_0 y Y_0 . Uno de los objetivos del *aprendizaje estadístico* (también conocido como *aprendizaje automático* o *machine learning*), es tratar de aproximar la función f_0 a partir de un conjunto de datos

$$\{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$$

dado. De manera más precisa, consideramos vectores aleatorios

$$\begin{aligned} X &= (X_1, X_2, \dots, X_n) \\ Y &= (Y_1, Y_2, \dots, Y_n) \end{aligned}$$

que representan los posibles valores de las observaciones independientes y dependientes, respectivamente, en una muestra dada. En este contexto, un *método de aprendizaje* consiste de una función $\hat{F}$, que depende de los vectores aleatorios X e Y , y que cuando se evalúa en los datos observados produce una función $\hat{f}$ que aproxima a la función desconocida f_0 .

Nota 2.0.1. En muchos modelos de aprendizaje estadístico es común tomar el vector $X = x$ como fijo, en cuyo caso la función $\hat{F}$ depende únicamente del vector aleatorio Y .

Para hacer estas ideas un poco más concretas, consideremos el caso particular en el que la función desconocida f_0 es una función lineal, es decir,

$$f_0(X_0) = b_0 + b_1 X_0$$

y supondremos además que $\epsilon \sim N(0, \sigma^2)$ es una variable aleatoria con distribución normal de media 0 y varianza σ^2 . Este contexto se conoce como el *modelo de regresión lineal simple* y es uno de los métodos más utilizados en aprendizaje estadístico. Notemos que en este caso nuestro objetivo es

encontrar una expresión que nos permita aproximar los parámetros desconocidos b_0 y b_1 en términos de los datos observados. Para ello consideraremos las expresiones que obtuvimos en la sección anterior para los coeficientes de la recta de mejor ajuste por mínimos cuadrados, y definiremos el *estimador de mínimos cuadrados* para los parámetros b_0 y b_1 como:

$$\hat{B}_1 = \frac{\sum_{i=1}^n (x_i - \bar{x})(Y_i - \bar{Y})}{\sum_{i=1}^n (x_i - \bar{x})^2} = \frac{\sum_{i=1}^n (x_i - \bar{x})Y_i}{\sum_{i=1}^n (x_i - \bar{x})^2}$$
$$\hat{B}_0 = \bar{Y} - \hat{B}_1 \bar{x}$$

Notemos que en estas expresiones estamos considerando el vector $X = x$ como fijo, mientras que el vector $Y = (Y_1, Y_2, \dots, Y_n)$ se sigue considerando como un vector aleatorio. En otras palabras, las fórmulas anteriores definen dos variables aleatorias $\hat{B}_0$ y $\hat{B}_1$ que dependen linealmente del vector aleatorio Y . Finalmente, definimos la función

$$\hat{F}(X_0) = \hat{B}_0 + \hat{B}_1 X_0$$

la cual se conoce como el *estimador de mínimos cuadrados* para la función f_0 . Notemos ahora que si evaluamos estos estimadores en el vector de datos observados $Y = y$, obtenemos los valores

$$\hat{\beta}_1 = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2} = \frac{\sum_{i=1}^n (x_i - \bar{x})y_i}{\sum_{i=1}^n (x_i - \bar{x})^2}$$
$$\hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}$$

que coinciden con los coeficientes de la recta de mejor ajuste por mínimos cuadrados que obtuvimos en la sección anterior. Por lo tanto, el estimador de mínimos cuadrados para la función f_0 evaluado en los datos observados es la recta de mejor ajuste por mínimos cuadrados:

$$\hat{f}(X_0) = \hat{\beta}_0 + \hat{\beta}_1 X_0$$

Para hacer este caso un poco más concreto, supongamos que fijamos los valores

$$b_0 = 0$$
$$b_1 = 1$$

y tomamos cinco valores para la variable independiente X_0 que se encuentren uniformemente distribuidos en el intervalo $[0, 10]$

```
set.seed(123)

n <- 5
x_data <- runif(n, min = 0, max = 10)
```

En particular, podemos considerar el vector

$$x = (2.88, 7.88, 4.09, 8.83, 9.4)$$

De acuerdo a nuestro modelo, los valores correspondientes de la variable dependiente Y_0 deberían de satisfacer una relación de la forma

$$Y_0 = b_0 + b_1 X_0 + \epsilon = X_0 + \epsilon$$

donde $\epsilon \sim N(0, \sigma^2)$. Supongamos que tomamos $\sigma^2 = 1$,

```
epsilon_data <- rnorm(n, mean = 0, sd = 1)
y_data <- x_data + epsilon_data

sample_data_tbl <- tibble(
  x = x_data,
  y = y_data
)
```

y consideremos el vector de observaciones

$$y = (1.19, 9.12, 3.98, 8.71, 9.59)$$

De manera gráfica, podemos visualizar los datos observados de la siguiente manera:

```
sample_data_plot <- sample_data_tbl %>%
  ggplot(aes(x = x, y = y)) +
  geom_point(
    color = c_pal("C red"),
    size = 1,
    alpha = 1
  ) +
  coord_fixed() +
  labs(
    title = "",
    x = "",
    y = ""
  ) +
  theme_krul()

sample_data_plot
```

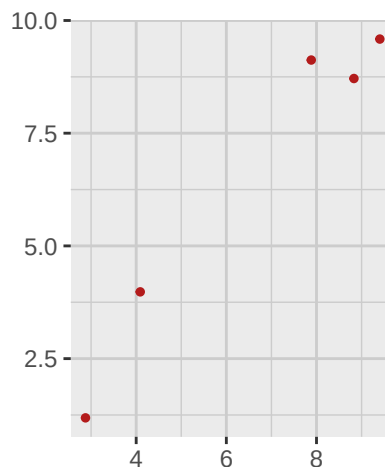


Figura 2.1: Diagrama de dispersión de los datos simulados.

Calculamos ahora

```
x_data_bar <- mean(x_data)
y_data_bar <- mean(y_data)
Sxx <- sum((x_data - x_data_bar)^2)
Sxy <- sum((x_data - x_data_bar) * y_data)
beta_1_hat <- Sxy / Sxx
beta_0_hat <- y_data_bar - beta_1_hat * x_data_bar
```

$$\bar{x} = 6.62$$

$$S_{xx} = \sum_{i=1}^n (x_i - \bar{x})^2 = 34.66$$

Podemos calcular ahora las fórmulas para los estimadores de mínimos cuadrados:

$$\hat{B}_1 = -0.11 Y_1 + 0.04 Y_2 + -0.07 Y_3 + 0.06 Y_4 + 0.08 Y_5$$

$$\hat{B}_0 = 0.91 Y_1 + -0.04 Y_2 + 0.68 Y_3 + -0.22 Y_4 + -0.33 Y_5$$

Se sigue que nuestro método de aprendizaje está dado por la función

$$\begin{aligned} \hat{F}(X_0) &= \hat{B}_0 + \hat{B}_1 X_0 \\ &= 0.91 Y_1 + -0.04 Y_2 + 0.68 Y_3 + -0.22 Y_4 + -0.33 Y_5 \\ &\quad + (-0.11 Y_1 + 0.04 Y_2 + -0.07 Y_3 + 0.06 Y_4 + 0.08 Y_5) X_0 \end{aligned}$$

Notemos que esta expresión depende no solamente de la variable aleatoria X_0 , sino también de las variables aleatorias $Y_1, Y_2, \dots, Y_5$. Evaluando ahora este estimador en los datos observados $Y = y$, obtenemos los valores

$$\hat{\beta}_1 = 1.24$$

$$\hat{\beta}_0 = -1.71$$

y la función estimada se convierte en:

$$\hat{f}(X_0) = -1.71 + 1.24 X_0$$

La cual se puede visualizar en la siguiente figura:

```
fitted_data_tbl <- sample_data_tbl %>%
  mutate(
    fitted = fitted(lm(y ~ x, data = .))
  )

fitted_data_plot <- fitted_data_tbl %>%
  ggplot(aes(x = x, y = y)) +
  geom_smooth(method = "lm", se = FALSE, color = c_pal("C blue")) +
  geom_segment(
```

```
    aes(xend = x, yend = fitted),
    color = c_pal("C green"),
    alpha = 1
) +
geom_point(color = c_pal("C red"), size = 1, alpha = 1) +
geom_point(
  aes(x = x, y = fitted),
  color = c_pal("C red"),
  size = 1,
  alpha = 1
) +
coord_fixed() +
labs(
  title = "",
  x = "",
  y = ""
) +
theme_krul()

fitted_data_plot
```

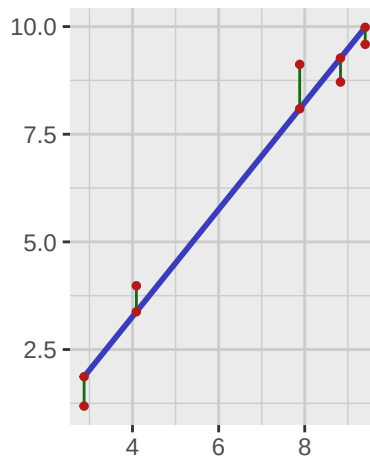


Figura 2.2: Línea recta que mejor se ajusta a los datos simulados.

Notemos que los valores estimados para los parámetros b_0 y b_1 no coinciden exactamente con los valores reales que utilizamos para simular los datos. En la siguiente sección analizaremos algunas propiedades estadísticas de estos estimadores que nos permitirán entender mejor su comportamiento y estimar la incertidumbre asociada a los mismos.

